

模指数运算+埃氏素性测试

基础题

密钥生成的核心步骤之一是找到两个大的素数。为了高效地找到这些素数，你需要实现一个经典的素数筛选算法，并将其与模指数运算结合，最终用 Shell 脚本自动化整个工作流。

第一部分：编程实现（C/C++）

你需要实现以下两个核心函数：

- 模指数运算（Modular Exponentiation）：实现 `long long mod_exp(long long a, long long e, long long m)` 函数，该函数应基于分治法（或称平方求幂法），用于高效计算 $a^e \bmod m$ 。
- 埃拉特罗切筛选法（Sieve of Eratosthenes）：实现 `void sieve(int n, bool primes[])` 函数，该函数使用埃拉特罗切筛选法来找出从 2 到 n 的所有素数。
- 主程序：编写 `main` 函数，接收整数 n 作为命令行参数。
- 调用你实现的 `sieve` 函数，找到 2 到 n 范围内的所有素数。
- 使用 `mod_exp` 函数对这些素数 p 进行简单的验证（例如，随机选择整数 a，计算 $a^p \bmod p$ 结果应为 a ，费马小定理），并确保 `mod_exp` 函数工作正常。
- 将所有找到的素数按从小到大的顺序输出到控制台，每个素数占一行。

第二部分：Git 版本控制

- 初始化仓库：在你的项目根目录执行 `git init`，并完成你的第一个提交，提交信息为 “Initial commit”。
- 创建分支：从 `main` 分支切出一个新的特性分支，命名为 `feat/crypto-primitives`。
- 分步提交：在新分支上分三次提交你的代码：
 - 第一次提交：提交 `mod_exp` 函数的实现，提交信息为 “feat: implement modular exponentiation function”。
 - 第二次提交：提交 `sieve` 函数的实现，提交信息为 “feat: implement Sieve of Eratosthenes algorithm”。
 - 第三次提交：提交 `main` 函数，完成所有代码，提交信息为 “feat: integrate components and add main program”。
- 推送并合并：将 `feat/crypto-primitives` 分支推送到远程仓库的 `main` 分支。

第三部分：Shell 脚本自动化

为了简化构建、测试和运行流程，你需要编写一个名为 `run.sh` 的 Shell 脚本来自动执行以下任务：

- 编译程序：使用 `g++` 编译器编译你的 C++ 源文件。
- 运行程序：如果编译成功，脚本将运行你的可执行文件，并传入一个合适的参数（例如 1000），将 1000 以内的所有素数输出到控制台。
- 错误处理：如果编译失败，脚本应立即停止并输出清晰的错误信息。

你的任务是：

- 完成第一部分的编程实现。
- 完成第二部分的 Git 操作，并清晰地记录每一个命令。
- 编写一个完整的 Shell 脚本，命名为 `run.sh`，实现第三部分的所有要求。

进阶题

你需要实现一个支持大整数运算的库，并在其基础上实现核心算法。

- 大整数类：

实现一个 `BigInt` 类/结构体，能够存储和操作任意长度的非负整数。你的类至少需要支持以下功能：

 - 构造函数（从 `string` 或 `long long` 对大整数赋值）
 - 加法、减法、乘法、除法和模运算
 - 模乘运算：实现高效的 $(a * b) \% m$ 运算，避免中间结果溢出。
 - 比较运算：例如 `>`, `<`, `==`。
- 大整数模指数运算：

使用你实现的 `BigInt` 类，实现 `BigInt mod_exp(BigInt a, BigInt e, BigInt m)` 函数。该函数应基于分治法，但现在要处理的是超大整数。